11 MODELLIZZAZIONE EFFICACE

OBIETTIVI

- Introdurre il **Unified Modelling Language (UML)**.
- Discutere come modellare vari elementi, in particolare:
  - **Entità**, come classi, pacchetti, nodi e componenti.
  - **Relazioni**, come dipendenze e associazioni.
  - **Processi**, come cambiamenti di stato e flussi di lavoro.
  - **Interazione dell'utente** con il sistema.
- Fornire consigli generali sulla creazione e sull'uso dei modelli.
- Spiegare come i professionisti del software usano la modellizzazione.

RIEPILOGO

La modellizzazione è stata introdotta nel **Capitolo 4**.

Nel **Capitolo 4**, abbiamo visto che le **astrazioni** sono utili per rappresentare parti di una soluzione, ma possono essere difficili da comprendere. I **modelli** forniscono un modo più concreto per lavorare con le astrazioni.

Un **modello** rappresenta entità e relazioni all'interno di una soluzione, includendo solo i dettagli rilevanti in un determinato contesto.

Tipologie di modelli

1. **Modelli statici** → Rappresentano un sistema in un momento specifico.
2. **Modelli dinamici** → Mostrano come il sistema cambia nel tempo.

Vantaggi dei modelli

✅ Riduzione dello sforzo mentale necessario per comprendere una soluzione.
✅ I modelli formali permettono di verificare se vengono violate delle regole.
✅ Alcuni modelli eseguibili prevedono il comportamento di una soluzione.

Diversi settori utilizzano modelli in modi differenti:

- **Ingegneri** → Modelli fisici di sistemi reali.
- **Scienziati** → Modelli matematici che riducono fenomeni a variabili.
- **Sviluppatori software** → Modelli concettuali per strutture dati, processi aziendali e architetture di rete.

L'**UML** (Unified Modelling Language) è il linguaggio di modellizzazione più utilizzato per descrivere sistemi software.

ENTITÀ

Le **entità** sono elementi di un sistema e vengono rappresentate nei **diagrammi statici** di UML.

Classi

Un **diagramma delle classi** suddivide una soluzione in classi, mostrando le loro proprietà e operazioni.

Esempio di una semplice classe UML:

python

CopiaModifica

```
class Vehicle:
    pass
```

UML la rappresenta così:

diff

CopiaModifica

```
+----------------+
|  Vehicle     |
+----------------+
```

Possiamo aggiungere **attributi** e **metodi** per rendere il modello più informativo:

lua

CopiaModifica

```
+----------------------+
|  Vehicle         |
+----------------------+
| model: string      |
| mileage: integer    |
| category: string    |
+----------------------+
```

```
| update_mileage()   |
+----------------------+
```

Codice corrispondente in Python:

python

CopiaModifica

```
class Vehicle:
    def __init__(self, model, mileage, category):
        self.model = model
        self.mileage = mileage
        self.category = category

    def update_mileage(self, new_mileage):
        self.mileage = new_mileage
```

📌 **Nota**: Anche se Python è **tipizzato dinamicamente**, nei diagrammi UML è utile specificare i tipi di dati per maggiore chiarezza.

Pacchetti e moduli

I **pacchetti** in UML rappresentano moduli o cartelle che raggruppano classi correlate.

Esempio:

diff

CopiaModifica

```
+-------------------+
| Vehicle Package  |
+-------------------+
| Vehicle        |
| Car            |
| Motorcycle     |
+-------------------+
```

In Python, un pacchetto corrisponde a una cartella con un file __init__.py che contiene più moduli.

RELAZIONI

Le entità di un sistema sono collegate tra loro da diverse **relazioni UML**:

1. **Dipendenze** → Un'entità dipende da un'altra.
2. **Associazioni** → Un'entità è legata a un'altra in modo diretto.

Esempio di dipendenza

L'oggetto EngineFactory dipende dall'oggetto Battery per funzionare.

python

CopiaModifica

```
class Battery:
  pass


class EngineFactory:
  def __init__(self, battery):
    self.battery = battery
```

📌 UML rappresenta questa relazione con una **freccia tratteggiata**.

Ereditarietà

Se una classe deriva da un'altra, eredita i suoi metodi e attributi.

Esempio in Python:

python

CopiaModifica

```
class Vehicle:
  pass


class Car(Vehicle):
  pass
```

📌 UML rappresenta l'ereditarietà con una **freccia piena** che punta alla classe madre.

PROCESSI

I modelli visti finora sono **statici**, ma in molti casi è utile modellare **cambiamenti nel tempo**.

Diagramma degli stati

Esempio: il **tornello della metropolitana** ha due stati: **bloccato** e **sbloccato**.

lua

CopiaModifica

```
+-------------+     Insert Coin     +------------+
|  Locked    | -------------------> |  Unlocked  |
+-------------+                     +------------+
```

In UML:

- Un **cerchio nero** rappresenta lo stato iniziale.
- Un **rettangolo con bordi arrotondati** rappresenta uno stato.
- **Freccia etichettata** → transizione di stato.

Diagramma delle attività

Se vogliamo modellare **un intero processo**, usiamo un **diagramma delle attività**.

Esempio: **Prelievo di denaro da un bancomat**

1. L'utente inserisce la carta.
2. La banca verifica il PIN.
3. Se corretto, permette il prelievo.
4. Restituisce la carta e i soldi.

INTERAZIONE UTENTE (USE CASES)

I **diagrammi dei casi d'uso** mostrano **chi può fare cosa** in un sistema.

Esempio: **Sistema di gestione studenti**

diff

CopiaModifica

```
+---------------------+
| Studente            |
+---------------------+
```

| - Visualizza voti |

| - Aggiorna dati |

+----------+

📌 Un **attore (stick figure)** rappresenta il tipo di utente.
📌 **Ellissi** rappresentano le azioni disponibili.

CONSIGLI GENERALI

Un modello efficace deve essere:

✅ **Astratto** → Nasconde i dettagli irrilevanti.
✅ **Comprensibile** → Più facile da leggere del codice.
✅ **Accurato** → Deve rappresentare la realtà.
✅ **Predittivo** → Permette di prevedere problemi futuri.
✅ **Economico** → Deve essere utile senza richiedere troppo tempo.

📌 **Non creare modelli inutili** → Un modello è utile solo se aiuta a **comunicare un'idea chiara**.

RIASSUNTO

- **UML** è uno strumento potente per modellare software.
- **Esistono modelli statici e dinamici** per descrivere strutture e processi.
- **Diagrammi delle classi**, **stati** e **casi d'uso** aiutano a visualizzare una soluzione prima di scrivere codice.
- **I professionisti usano UML in modo flessibile**, adattandolo alle esigenze del progetto.